

- 2023.7.17: 该技巧目前已知的最早来源: [skip2004](#)。
- 2024.7.6: 修订文章。

DFS 序求 LCA 无论是在时间常数, 空间常数还是好写程度均吊打欧拉序。

定义

DFS 序表示对一棵树进行深度优先搜索得到的 **结点序列**, 而 **时间戳** dfn 表示每个结点在 DFS 序中的位置。需要区分这两个概念。

算法介绍

考虑树上的两个结点 u, v 及其最近公共祖先 d , 使用欧拉序而不是 DFS 序求 LCA 的原因是在欧拉序中, d 在 u, v 之间出现过, 但在 DFS 序中, d 没有在 u, v 之间出现过。

DFS 序的性质: 祖先先于后代遍历, 即若 u 是 v 的祖先, 则 $dfn_u < dfn_v$ 。

不妨设 $dfn_u < dfn_v$, 那么 v 不是 u 的祖先。

- 当 u 不是 v 的祖先时, DFS 的顺序为从 d 下降到 u , 再回到 d , 再下降到 v 。因为到达 u 在到达 d 之后, 而到达 v 在离开 d 之前, 所以 u, v 的 DFS 序之间的所有结点一定落在 d 的子树内 (不含 d)。

考察 d 在 v 方向上的第一个结点 v' , 即设 v' 为 d 的「子树包含 v 的」儿子。根据 DFS 的顺序, v' 一定在 u, v 的 DFS 序之间。这说明只需求 u, v 的 DFS 序之间深度最小的任意结点, 那么 **它的父亲** 即为 u, v 的 LCA。换言之, 在 u, v 的 DFS 序之间一定存在 d 的儿子。

- u, v 形成祖先-后代关系的情况容易判断, 但不优美, 因为还需记录每个结点的子树大小, 不能体现出 DFS 求 LCA 的优势: 简洁。

此时 u 一定是 v 的祖先。考虑令查询区间从 $[dfn_u, dfn_v]$ 变成 $[dfn_u + 1, dfn_v]$ 。对于情况 1, $u \neq v'$, 所以情况 2 对于算法进行的修改仍适用于情况 1。

综上, 若 $u \neq v$, 则它们的 LCA 等于 DFS 序上位置在 $[dfn_u + 1, dfn_v]$ 的深度最小的任意结点的父亲。若 $u = v$, 则它们的 LCA 就等于 u , 这是唯一需要特判的情况。

一种避免记录每个结点的父亲和深度的方法是直接在 ST 表的最底层记录父亲, 比较时取时间戳较小的结点。如果你完全理解了 DFS 序求 LCA, 自然能够理解这个技巧的正确性。

预处理 ST 表的复杂度仍为 $\mathcal{O}(n \log n)$, 但常数减少一半。以下是模板题 [P3379](#) 的代码。

```

#include <bits/stdc++.h>
using namespace std;
constexpr int N = 5e5 + 5;
int n, m, R, dn, dfn[N], mi[19][N];
vector<int> e[N];
int get(int x, int y) {return dfn[x] < dfn[y] ? x : y;}
void dfs(int id, int f) {
    mi[0][dfn[id] = ++dn] = f;
    for(int it : e[id]) if(it != f) dfs(it, id);
}
int lca(int u, int v) {
    if(u == v) return u;
    if((u = dfn[u]) > (v = dfn[v])) swap(u, v);
    int d = __lg(v - u++);
    return get(mi[d][u], mi[d][v - (1 << d) + 1]);
}
int main() {
    scanf("%d %d %d", &n, &m, &R);
    for(int i = 2, u, v; i <= n; i++) {
        scanf("%d %d", &u, &v);
        e[u].push_back(v), e[v].push_back(u);
    }
    dfs(R, 0);
    for(int i = 1; i <= __lg(n); i++)
        for(int j = 1; j + (1 << i) - 1 <= n; j++)
            mi[i][j] = get(mi[i - 1][j], mi[i - 1][j + (1 << i - 1)]);
    for(int i = 1, u, v; i <= m; i++) scanf("%d %d", &u, &v), printf("%d\n", lca(u, v));
    return 0;
}

```

和各种 LCA 算法的对比

对比 DFS 序和欧拉序，不仅预处理的时间常数减半（欧拉序 LCA 的瓶颈恰在于预处理，DFS 是线性），空间常数也减半（核心优势），而且更好写（对于很多题目不需要再同时求欧拉序和 DFS 序），也无需担心忘记开两倍空间。

对比 DFS 序和倍增，前者单次查询复杂度更优。

对于 DFS 序和四毛子，前者更好写。

对于 DFS 序和树剖，前者更好写，且单次查询复杂度更优。不过树剖常数较小，如果求 LCA 不是瓶颈且其它部分需要使用树剖，则树剖 LCA 也是不错的选择。

将 DFS 序求 LCA 发扬光大，让欧拉序求 LCA 成为时代的眼泪！